

TriCube: An Experimental Geometric Hash and XOF Candidate Based on Tetrahedral State Evolution

Steven Reid

2026

Abstract

TriCube is an experimental hash, extendable-output function, and deterministic stream generator based on cube-connected state evolution. The current construction uses a 2048-bit state arranged as a $3 \times 3 \times 3$ vertex grid plus shell lanes. The eight cube cells of the grid are decomposed into tetrahedra, and each round combines local tetrahedral ARX mixing, edge-coupled propagation, shell coupling, and global lane permutation. This paper documents the public construction, implementation status, test vectors, early statistical evidence, known limitations, and the work required before any security claim would be responsible. TriCube is not presented as cryptographically secure or production ready.

1. Introduction

TriCube grew out of a series of notebook and local-repository experiments on recursive geometric mixing, entropy diagnostics, and byte-stream generators. Several earlier candidates used cube or tetrahedron language but did not explicitly model a tetrahedral state topology. The current version was separated from that broader archive into a narrower project: a concrete geometric hash/XOF candidate with standalone C code, Python reference code, fixed vectors, and reproducible tests.

The purpose of this release is not to claim a new secure hash. The purpose is to make the construction inspectable enough for review. The repository removes unsupported claims, keeps the implementation deterministic, and records both positive and negative evidence.

2. Design Goals

The design goals are:

1. Preserve the geometric TriCube idea in an explicit state model.
2. Avoid depending on SHA, BLAKE, or another mature hash as the internal primitive.
3. Provide fixed hash and XOF behavior with deterministic test vectors.
4. Provide a stream mode for statistical testing.
5. Keep the implementation small enough to audit.
6. State limitations directly.

TriCube is not designed as a drop-in replacement for SHA-2, SHA-3/SHAKE, BLAKE2, or BLAKE3.

3. Construction

The TriCube state contains 32 64-bit lanes. Lanes 0 through 26 map to vertices in a $3 \times 3 \times 3$ grid. The mapping is:

$$\text{index}(x, y, z) = x + 3 * (y + 3 * z).$$

The grid contains eight cube cells. Each cube cell is decomposed into six tetrahedra:

```
(0, 1, 3, 7)
(0, 3, 2, 7)
(0, 2, 6, 7)
(0, 6, 4, 7)
(0, 4, 5, 7)
(0, 5, 1, 7)
```

Across all cube cells this gives 48 tetrahedral groups. A round applies orientation-dependent ARX mixing to these groups, then couples adjacent grid vertices, then mixes shell lanes, then applies a deterministic lane permutation.

4. Modes

Hash mode emits a 256-bit digest. XOF mode emits an arbitrary number of bytes. Stream mode initializes the state from a seed and emits deterministic bytes for external statistical batteries. The stream mode is a test interface, not a random-number generator claim.

5. Implementation

The repository contains a C11 implementation with a public header, CLI, Makefile, CMake file, and C tests. It also contains a Python package with tricube.hash, tricube.hexdigest, tricube.xof, and tricube.stream. The Python path is a reference implementation for scripting and vector checks; the C path is the primary performance path.

6. Test Vectors

The public 256-bit digest vectors are:

Message	Digest
empty string	7fcaaa35165277bcaca583e23ef1d3545705e14d39f3ed7a802b1275d920cf49
abc	779403a9c748fc3213493953fc17309367b37161c00dc19059c14db63774e11e
TriCube	5b4c461fe975dfba72fc9b2fbcf04e3a1a807c83fa4502c238ee4fe48b736d13

7. Current Evidence

The May 2026 validation pass found that the C stream path was deterministic and much faster than the Python reference path. In the saved 1 MiB internal check, the C stream measured about 44.3 MiB/s on an Apple M4 Pro Mac mini. Basic entropy, bit-balance, serial-correlation, and avalanche sanity checks did not show obvious failure at that size.

Saved external artifacts include a limited Dieharder pass, a TestU01 SmallCrush pass, and a PractRand 1 GiB expanded run classified as WARN. The PractRand run reached 1 GiB and ended with no final-level anomalies, but it reported earlier low-bit suspicious and unusual results. That warning is the most important current test finding.

8. Limitations

TriCube has no proof of collision resistance, preimage resistance, second-preimage resistance, indifferentiability, or output pseudorandomness. No independent cryptanalysis has been completed. Reduced-round analysis, differential trails, rotational behavior, algebraic analysis, birthday collision campaigns, state-recovery attempts, and side-channel review are all incomplete.

The C implementation is portable and usable, but not optimized like mature hash libraries. Throughput is not currently competitive with optimized SHA-2, SHA-3, BLAKE2, or BLAKE3 implementations.

9. Comparison to Known Work

TriCube shares broad patterns with sponge and ARX designs: absorb input, permute state, and squeeze output. Those ideas are established. The candidate contribution is the explicit tetrahedral/cube-connected state evolution. This should be compared against SHA-2, SHA-3/SHAKE, BLAKE2, BLAKE3, CubeHash, Keccak-family permutations, Xoodoo/Xoodyak-like permutations, and KangarooTwelve before any novelty claim is made.

10. Reproducibility

The public repository is intended to be reproducible from source:

```
make -C c test
python -m pip install -e ".[test]"
pytest -q
python benchmarks/bench_stream.py --bytes 1048576
```

External batteries can be run with the scripts in tools/, provided the tools are installed locally.

11. Conclusion

TriCube is a concrete experimental candidate, not a secure primitive. The strongest current result is that the geometric construction has been made deterministic, implemented in standalone C, wrapped in Python, and tested enough to identify the next hard questions. The main blocking issue is cryptanalytic evidence, especially the PractRand low-bit warning and the absence of reduced-round and differential analysis.

Declarations

This work received no external funding. The author reports no external financial conflict of interest related to this repository. AI assistance was used to organize code, documentation, tests, and manuscript material. The design, claims, limitations, and interpretation remain the responsibility of the author.

Code availability: <https://github.com/RRG314/tricube-hash>